

# Sydney Housing Price Prediction and Decision Support System

## 8.1D task

### Part 1 : Problem Definition & Data Collection

The problem. A real-estate agency wants a tool that estimates a Sydney property's likely sale price from its characteristics, to support an agent's appraisal rather than replace it. Target: `sale_price` (AUD). Inputs: property features from public sold listings.

Suburbs (chosen for contrasting markets):

- **Mosman** : blue-chip lower-north-shore; high median, wide spread (harbourside premiums).
- **Parramatta** : dense, apartment-heavy metropolitan centre; mid-market.
- **Blacktown** : more affordable outer-west; house-and-land mix.

I chose these suburbs to compare different buying options. Blacktown represents a more affordable option in my sample, which would appeal to me if keeping the purchase cost manageable was the main priority. Parramatta offers an apartment-focused alternative that I would consider if I preferred a smaller property over a house. Mosman represents the higher-budget option, with a much wider price range and several expensive houses. I would consider it if my budget allowed a more premium property. Together, these suburbs let me investigate how location and property characteristics affect prices across different markets.

Data was transcribed from public Domain.com.au sold listings. The dataset contains 102 properties individually transcribed from publicly available Domain sold listings. The recorded information included address, suburb, advertised sale price, sale date, property type, bedrooms, bathrooms and car spaces. Unavailable values were left missing rather than guessed.

| Suburb     | Properties | Median price | Price range       |
|------------|------------|--------------|-------------------|
| Mosman     | 35         | \$1,600,000  | \$610k – \$11.65M |
| Blacktown  | 34         | \$854,000    | \$375k – \$1.90M  |
| Parramatta | 33         | \$580,000    | \$250k – \$800k   |

Note on dropped fields: Land size and advertised floor area were published for only ~17% and ~30% of listings respectively, and build year was almost never shown, so those columns were dropped rather than imputed from near-nothing. Where present, the area figures were advertised rather than independently verified. Two singleton categories (Studio, Semi-detached) were folded into their nearest type (Apartment, House). The honest consequence: the model leans on bedrooms, bathrooms, car spaces, suburb and property type — a deliberately lean feature set.

## Part 2 : Data Understanding & Feature Engineering

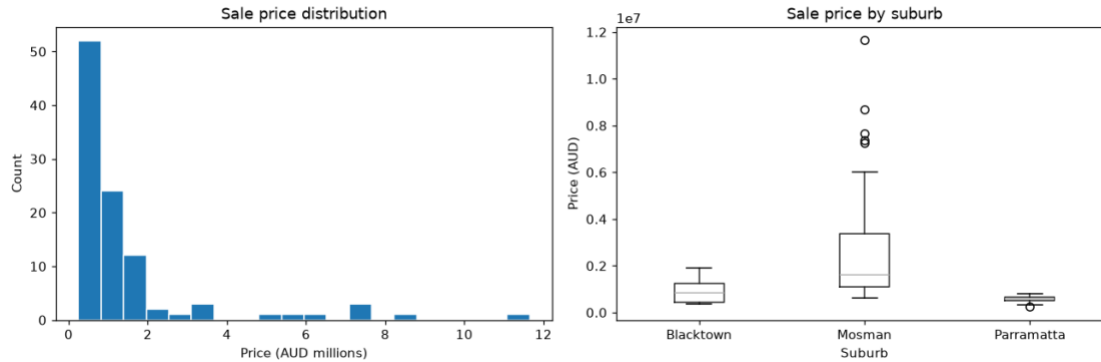


Figure 1. Distribution of sale prices, and sale price by suburb.

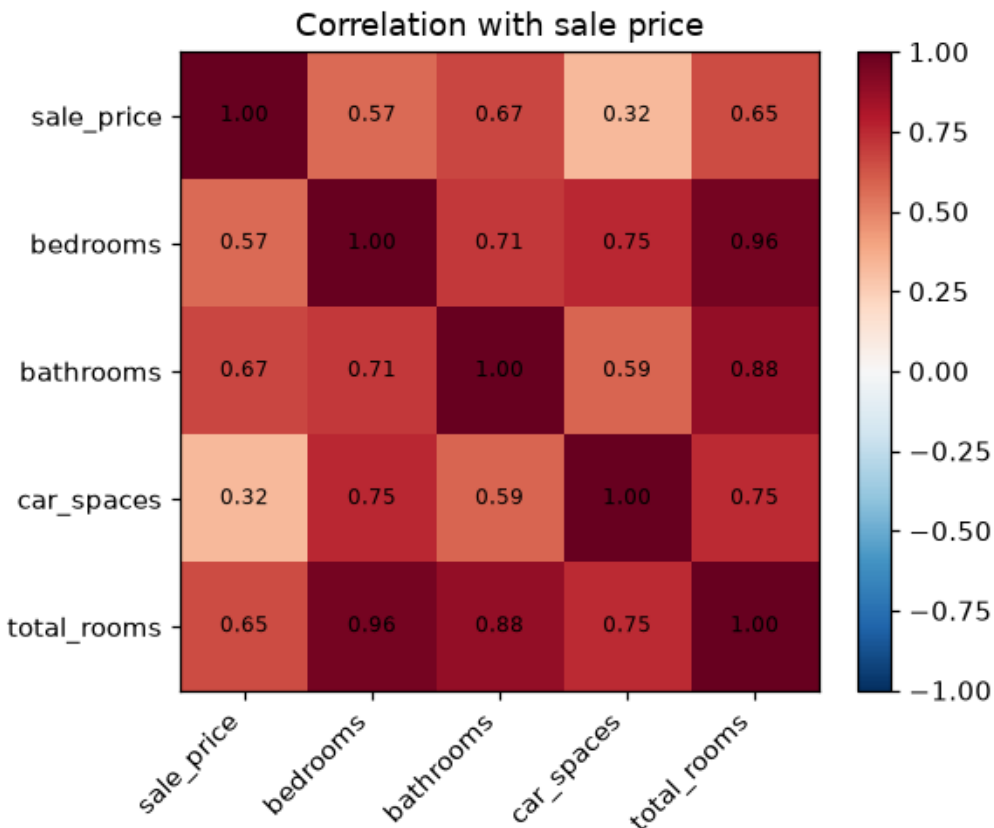


Figure 2. Correlation of numeric features with sale price.

Before modelling, I expected suburb, bedrooms and bathrooms to have the greatest influence on sale price. Suburb seemed important because location can affect what buyers are willing to pay, and the sample medians differed considerably: \$1.60 million in Mosman, \$854,000 in Blacktown and \$580,000 in Parramatta.

I expected bedrooms to matter because they indicate accommodation capacity, while bathrooms may reflect a property's size and convenience. The exploratory results

supported these expectations: bedrooms had a correlation of 0.57 with sale price, and bathrooms had a stronger correlation of 0.67. However, these relationships do not prove which features the models will rely on most. After training, I would compare this expectation with model-based feature importance.

Note the very narrow sale window (Jun–Sep 2026): there's essentially no time trend to exploit, so `sale_date` is used for description only, not as a model feature — worth stating as a limitation.

### Part 3 : Model Development & Evaluation

Three models from different families; cross-validation is run on the development rows only (the 10 holdout rows are reserved for Part 5).

| Model                    | Family               | Rationale                                                     |
|--------------------------|----------------------|---------------------------------------------------------------|
| <b>Ridge</b>             | Linear (regularised) | Interpretable baseline; assumes additive, near-linear effects |
| <b>Random Forest</b>     | Bagging ensemble     | Captures non-linearity & interactions; robust to outliers     |
| <b>Gradient Boosting</b> | Boosting ensemble    | Often strongest on tabular data; fits residuals sequentially  |

Before training, I expected Random Forest to perform best. Property prices may depend on combinations of features. for example, an extra bedroom could add different amounts of value in different suburbs. Random Forest can capture these interactions and non-linear relationships. Averaging many trees may also help it generalise better than Gradient Boosting on this small dataset. I expected Ridge Regression to provide a useful baseline, but its linear structure might struggle with the wide price differences. I would check this prediction using the five-fold cross-validation results.

| Model             | CV R <sup>2</sup> | CV RMSE | CV MAE | Train R <sup>2</sup> | Overfit gap |
|-------------------|-------------------|---------|--------|----------------------|-------------|
| Ridge             | 0.56              | \$1.16M | \$716k | 0.66                 | 0.11        |
| Random Forest     | 0.72              | \$0.89M | \$403k | 0.95                 | 0.23        |
| Gradient Boosting | 0.67              | \$1.00M | \$431k | 1.00                 | 0.32        |

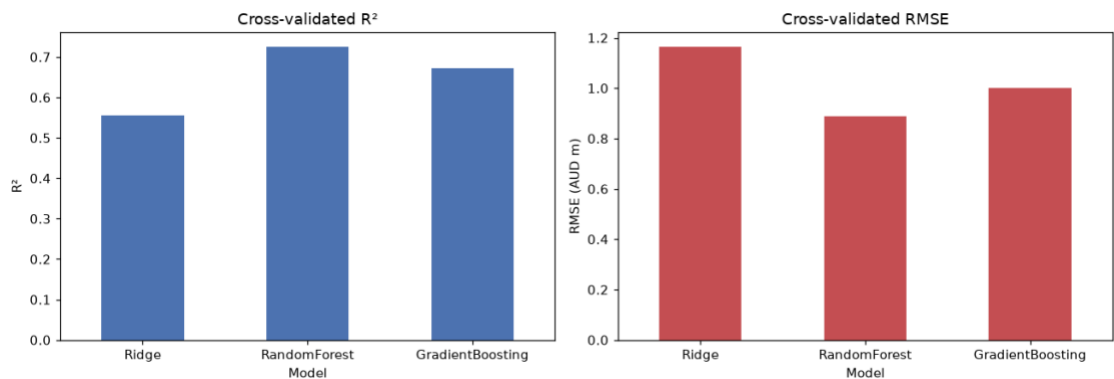


Figure 3. Cross-validated  $R^2$  and RMSE by model.

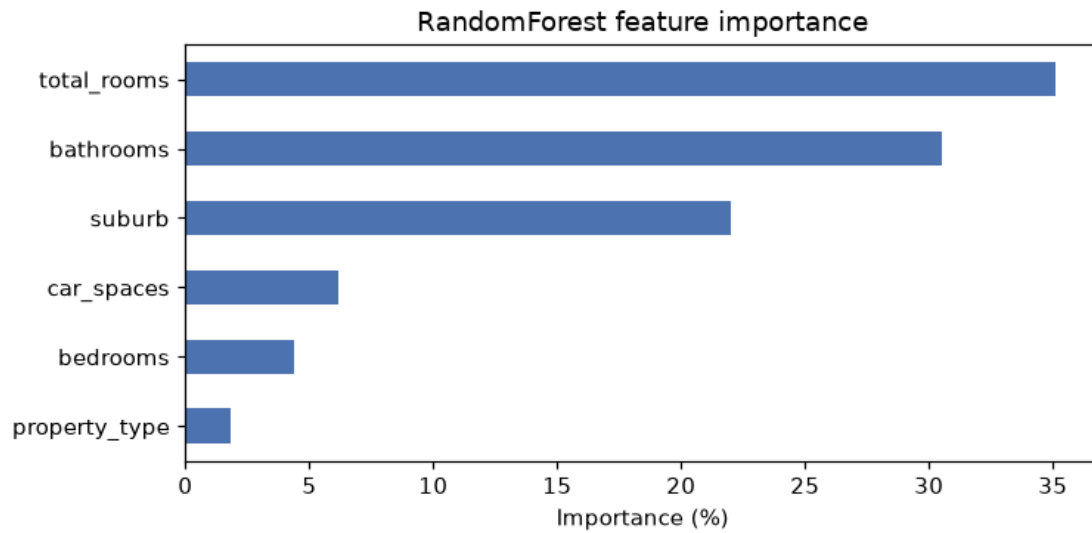


Figure 4. Random Forest feature importance.

Revisiting my feature expectations. Before modelling I expected suburb, bedrooms and bathrooms to matter most. The Random Forest importances partly support this: bathrooms (30%) and suburb (22%) are both in the top three, but bedrooms was not, on its own it contributes only about 4%. Instead, the engineered total-rooms feature (35%) is the strongest predictor. This suggests that overall property size, captured by combined room count, carries more signal than the bedroom count alone, and that a single bathroom figure is a better quality/scale proxy than I anticipated.

### Underfitting, overfitting and model recommendation

Random Forest performed best, with a cross-validation  $R^2$  of 0.72, RMSE of approximately \$889,000 and MAE of \$403,000. This supports my prediction that Random Forest would outperform the other models.

Both tree ensembles showed signs of overfitting. Random Forest's training  $R^2$  was 0.95, compared with 0.72 in cross-validation. Gradient Boosting had an even larger gap, falling from approximately 1.00 during training to 0.67 in cross-validation. This suggests that it fitted the training properties very closely but generalised less successfully.

Ridge Regression achieved a training  $R^2$  of 0.66 and cross-validation  $R^2$  of 0.56. Its smaller gap suggests less overfitting, while its weaker performance on both sets is consistent with underfitting. Its linear structure may not capture the interactions between location and property characteristics.

I recommend Random Forest because it achieved the strongest cross-validation results across all three metrics. However, the substantial errors mean it should support an agent's appraisal rather than replace it. Ridge remains worth considering when interpretability is a priority, although these results do not establish that it is more stable. With only 92 development properties, additional data and validation would be needed before

deployment. A further limitation is that standard k-fold cross-validation can place separate units from the same building into both training and validation folds (16 of the 102 records share a building address), which may make the cross-validated scores slightly optimistic; a grouped split by building would be a more conservative approach.

## Part 4 : Investigating Prediction Failures

Using out-of-fold predictions on the development set (every row predicted by a model that didn't see it), I isolate the five largest absolute errors.

| Property                             | Actual   | Predicted | Error   |
|--------------------------------------|----------|-----------|---------|
| 21 Morella Rd, Mosman (4bd/4ba)      | \$11.65M | \$6.584M  | -43.5%  |
| 3 Gordon St, Mosman (5bd/2ba)        | \$7.65M  | \$3.576M  | -53.3%  |
| 13 Lancaster St, Blacktown (7bd/4ba) | \$1.90M  | \$5.111M  | +169.0% |
| 105 Holt Ave, Mosman (4bd/3ba)       | \$3.55M  | \$6.094M  | +71.7%  |
| 5 Botanic Rd, Mosman (4bd/3ba)       | \$8.70M  | \$6.452M  | -25.8%  |

### Analysis of the five largest prediction errors

1. **21 Morella Road, Mosman:** This four-bedroom, four-bathroom house with two car spaces sold for **\$11.65 million**, but the model predicted **\$6.584 million**. It under-predicted by **\$5.066 million (43.5%)**. Unrecorded features such as views, condition or a premium position could help explain the difference. The limited number of expensive properties also makes this end of the market difficult to learn.
2. **3 Gordon Street, Mosman:** This five-bedroom, two-bathroom house with two car spaces sold for **\$7.65 million**, compared with a prediction of **\$3.576 million**. The under-prediction was **\$4.074 million (53.3%)**. Its room counts may not capture the property's overall quality or location within Mosman. Land size, renovation quality and views are possible missing influences.
3. **13 Lancaster Street, Blacktown:** This seven-bedroom, four-bathroom house with six car spaces sold for **\$1.90 million**, but was predicted at **\$5.111 million**. The model over-predicted by **\$3.211 million (169.0%)**. Its unusually large room and parking counts may have placed it alongside expensive houses in the model's decision rules. With few comparable Blacktown properties, the model may not have learned how these features interact with suburb.
4. **105 Holt Avenue, Mosman:** This four-bedroom, three-bathroom house had an actual price of **\$3.55 million** and a predicted price of **\$6.094 million**. The over-prediction was **\$2.544 million (71.7%)**. Car spaces were missing and therefore imputed. Other unrecorded differences, such as condition, land size or position, could distinguish it from higher-priced houses with similar room counts.
5. **5 Botanic Road, Mosman:** This four-bedroom, three-bathroom house with two car spaces sold for **\$8.70 million**, while the model predicted **\$6.452 million**. It under-predicted by **\$2.248 million (25.8%)**. The model could not account for potential

architectural quality, renovations or a particularly desirable position, which may contribute value beyond the recorded features.

These explanations are plausible hypotheses rather than confirmed causes. All five errors involved houses, and four were in Mosman, suggesting difficulty with expensive or unusual properties. Predictions should therefore be treated cautiously for prestige homes and properties unlike the training sample. An agent should check the listing's condition, land, views and local context before relying on the estimate.

## Part 5 : Human Judgement vs Machine Learning vs LLM

The 10 pre-tagged **holdout** properties are the held-out test set. I compare three estimates against the true price:

1. **ML** : best model (trained on all development data) , filled below.
2. **LLM** : paste each property's details into ChatGPT, record its estimate.
3. **Human** : my own estimate.

| Property                                         | Suburb     | Actual      | ML          | LLM         | Human       |
|--------------------------------------------------|------------|-------------|-------------|-------------|-------------|
| 42 Burke Street                                  | Blacktown  | \$1,260,000 | \$1,992,000 | \$1,450,000 | \$1,320,000 |
| 41 Dagmar Crescent                               | Blacktown  | \$880,000   | \$1,203,000 | \$1,000,000 | \$900,000   |
| 45 & 45a Dina Beth Avenue                        | Blacktown  | \$1,250,000 | \$1,359,000 | \$1,350,000 | \$1,200,000 |
| 10/4 Peace Lane                                  | Parramatta | \$570,000   | \$640,000   | \$750,000   | \$600,000   |
| 5/30-32 Queens Avenue                            | Parramatta | \$680,000   | \$565,000   | \$650,000   | \$700,000   |
| 1417/32 Hunter Street (AKA 1417/45 Macquarie St) | Parramatta | \$560,000   | \$467,000   | \$500,000   | \$530,000   |
| 2/81A Glover Street                              | Mosman     | \$1,440,000 | \$1,720,000 | \$1,700,000 | \$1,500,000 |
| 16/5 The Esplanade                               | Mosman     | \$1,570,000 | \$796,000   | \$950,000   | \$1,000,000 |
| 14 Almora Street                                 | Mosman     | \$5,025,000 | \$5,636,000 | \$3,600,000 | \$5,000,000 |
| 11 Euryalus Street                               | Mosman     | \$7,260,000 | \$7,705,000 | \$6,500,000 | \$7,000,000 |

### Comparison of human, machine learning and LLM estimates

The human estimates were closest on all ten properties, with an MAE of **\$112,500** and MAPE of **6.9%**. Random Forest achieved an MAE of **\$355,200**, compared with **\$374,500** for the LLM. However, the LLM had a lower MAPE: **18.0%**, compared with **23.6%** for Random Forest. Therefore, the ML model performed better in average dollar terms, while the LLM performed better in average percentage terms.

Their strengths varied between properties. Random Forest was closer on **10/4 Peace Lane, Parramatta**, predicting \$640,000 against an actual price of \$570,000; the LLM estimated \$750,000. Conversely, the LLM was closer on **5/30-32 Queens Avenue**, estimating \$650,000 against \$680,000, while Random Forest predicted \$565,000. Random

Forest was also closer on both expensive Mosman houses, although both approaches over-valued or under-valued individual properties.

Both automated approaches substantially under-valued **16/5 The Esplanade, Mosman**. It sold for \$1.57 million, but Random Forest predicted \$796,000 and the LLM estimated \$950,000. This illustrates how suburb and room counts can leave important differences between properties unexplained.

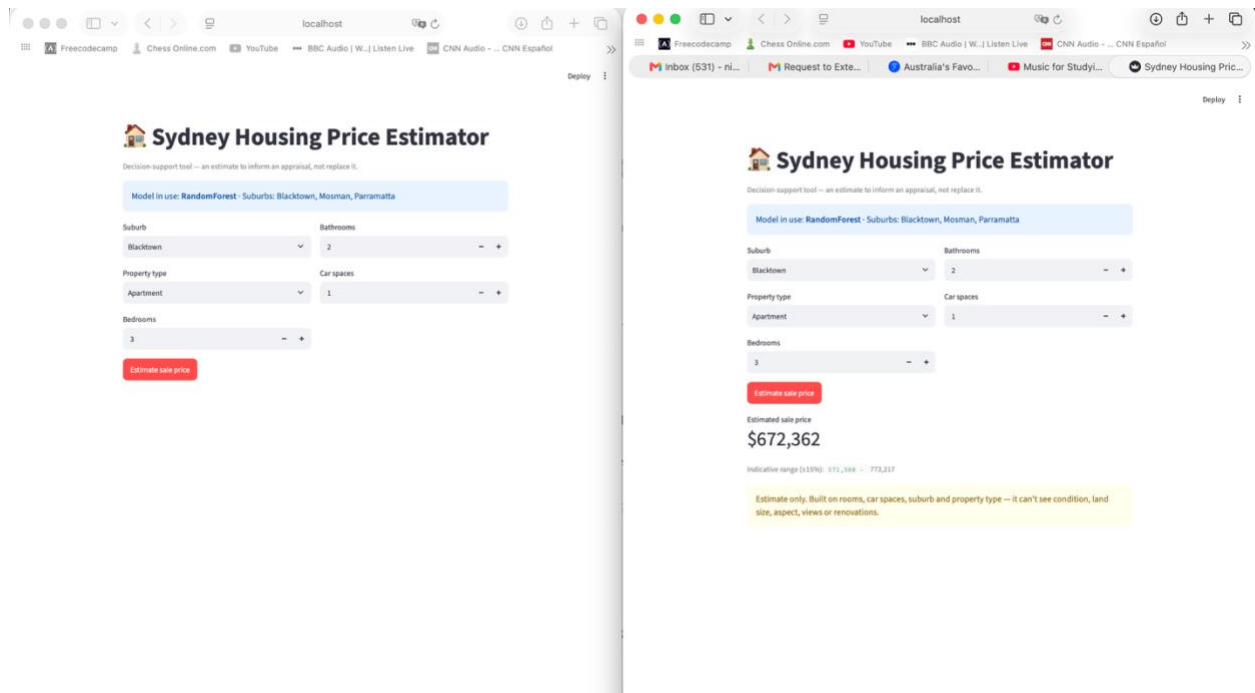
The comparison has two important limitations. My human estimates were made with the actual prices visible, so their apparent advantage cannot demonstrate superior unaided judgement. The LLM estimates were also obtained in an existing conversation where the dataset had already been discussed, so they cannot be treated as a fully independent blind test.

My view is that informed human judgement still adds value by considering condition, views, land and local context that the model does not capture. Automated estimates can provide a starting point, but unusual or expensive properties need closer assessment. A stronger experiment would use new properties, record human estimates before revealing prices, and obtain LLM estimates in a fresh conversation containing only the permitted features.

## Part 6 : Deployment & Reflection

The final model is refit on all 102 rows and saved for the Streamlit app (`app.py`).

**Using the application.** The app is a Streamlit web tool. To run it: activate the project environment (`conda activate sit799`), install the runtime packages if needed (`pip install streamlit joblib`), and from the project folder run `streamlit run app.py`. This opens the app at `http://localhost:8501`. The user selects a suburb and property type, enters the number of bedrooms, bathrooms and car spaces, and clicks Estimate sale price. The app loads the trained Random Forest model, runs the entered features through the same preprocessing pipeline used in training, and displays the estimated sale price with an indicative  $\pm 15\%$  range and a reminder that the estimate cannot account for condition, land size, aspect or views. For example, a three-bedroom Blacktown apartment with two bathrooms and one car space returns an estimate of approximately \$672,000.



## Reflection

This project showed me that the information available about a property can matter as much as the model used to predict its price. Collecting records from public sold listings involves inconsistent formats, missing fields and the risk of transcription errors. Land size, floor area and build year were not consistently available, so they were excluded from the modelling dataset. This simplified preprocessing but removed potentially useful explanations for price differences, leaving the models dependent on suburb, property type and room counts.

Random Forest achieved the strongest cross-validation results, with an  $R^2$  of approximately **0.72** and MAE of **\$403,000**. Its ability to capture non-linear relationships and feature interactions may explain its advantage over Ridge. However, its higher training performance indicated overfitting, and the large prediction errors limited its practical usefulness. Building the Streamlit app made deployment straightforward, but making a prediction easy to access does not make it reliable. The app should provide a starting estimate for an agent to review. Its displayed  $\pm 15\%$  range is illustrative rather than a validated prediction interval.

There are also ethical concerns. Suburb-based predictions can reproduce existing differences between locations, while an unrepresentative sample may produce uneven accuracy across property groups. Users could mistake an automated estimate for an objective valuation. Clear limitations, monitoring of errors across suburbs and property types, and human review are therefore important.

With more time, I would collect additional properties across more Sydney suburbs and a wider sale period. I would also prioritise condition, land and floor area, and finer location features. Listing descriptions could be explored through natural language processing, while

photographs might provide condition information through computer vision. Finally, I would repeat the human and LLM comparison under properly blinded conditions.

### **Data and Code Availability**

All data, source code, notebooks and the deployed application source are available in the project GitHub repository:

<https://github.com/nicolasai2026/sit720---8.1D-sydney-housing>

### **Acknowledgement of GenAI Use**

Generative AI tools (ChatGPT) were used during this project for part 5 LLM and brainstorming and editing my work for grammatical and spelling errors. All data was checked, the analysis was executed and verified by me, and the interpretations, decisions and reflections presented here are my own.

### **References**

Domain Group 2026, Sold listings – Mosman, Blacktown and Parramatta NSW, Domain.com.au, viewed September 2026, <https://www.domain.com.au>.

Harris, CR, Millman, KJ, van der Walt, SJ et al. 2020, 'Array programming with NumPy', Nature, vol. 585, pp. 357–362.

Hunter, JD 2007, 'Matplotlib: A 2D graphics environment', Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95.

McKinney, W 2010, 'Data structures for statistical computing in Python', Proceedings of the 9th Python in Science Conference, pp. 56–61.

OpenAI 2026, ChatGPT, viewed 18 September 2026, <https://chat.openai.com>.

Pedregosa, F, Varoquaux, G, Gramfort, A et al. 2011, 'Scikit-learn: Machine learning in Python', Journal of Machine Learning Research, vol. 12, pp. 2825–2830.

Streamlit Inc. 2026, Streamlit documentation, viewed September 2026, <https://docs.streamlit.io>.

## Appendix A: Notebook Source Code

### Cell 1

```
import warnings
from pathlib import Path
import numpy as np, pandas as pd, matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.model_selection import KFold, cross_validate, cross_val_predict
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib

warnings.filterwarnings("ignore")
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
FIG_DIR = Path("figures"); FIG_DIR.mkdir(exist_ok=True)
MODEL_DIR = Path("model"); MODEL_DIR.mkdir(exist_ok=True)
pd.set_option("display.float_format", lambda x: f"{x:,.2f}")
plt.rcParams["figure.dpi"] = 110
print("Setup complete.")
```

### Cell 2

```
DATA_PATH = Path("sydney_housing.csv")
assert DATA_PATH.exists(), "sydney_housing.csv not found – place it beside  
this notebook."
df = pd.read_csv(DATA_PATH, parse_dates=["sale_date"])
print(f"Loaded {df.shape[0]} rows, {df.shape[1]} columns")
df.head()
```

### Cell 3

```
print("Rows per suburb (need >=30 each):")
print(df["suburb"].value_counts(), "\n")
print("Split:", df["split"].value_counts().to_dict(), "\n")
print("Missing values:")
print(df.isna().sum(), "\n")
print(df[["bedrooms", "bathrooms", "car_spaces", "sale_price"]].describe().T)
```

### Cell 4

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.2))
axes[0].hist(df["sale_price"]/1e6, bins=20, edgecolor="white")
axes[0].set(title="Sale price distribution", xlabel="Price (AUD millions)",
ylabel="Count")
```

```

df.boxplot(column="sale_price", by="suburb", ax=axes[1], grid=False)
axes[1].set(title="Sale price by suburb", xlabel="Suburb", ylabel="Price (AUD)")
plt.suptitle(""); fig.tight_layout()
fig.savefig(FIG_DIR / "price_distribution.png", bbox_inches="tight")
plt.show()
print(df.groupby("suburb")["sale_price"].agg(["count", "median", "max"]))

```

## Cell 5

```

df["total_rooms"] = df["bedrooms"] + df["bathrooms"]
num = ["sale_price", "bedrooms", "bathrooms", "car_spaces", "total_rooms"]
corr = df[num].corr()

fig, ax = plt.subplots(figsize=(5.5, 4.6))
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
ax.set_xticks(range(len(num))); ax.set_xticklabels(num, rotation=45, ha="right")
ax.set_yticks(range(len(num))); ax.set_yticklabels(num)
for (i,j),v in np.ndenumerate(corr.values):
    ax.text(j,i,f"{v:.2f}", ha="center", va="center", fontsize=8)
ax.set_title("Correlation with sale price"); fig.colorbar(im, fraction=0.046)
fig.tight_layout(); fig.savefig(FIG_DIR / "correlations.png",
bbox_inches="tight"); plt.show()
print(corr["sale_price"].sort_values(ascending=False))

```

## Cell 6

```

dev = df[df["split"] == "development"].copy()
holdout = df[df["split"] == "holdout"].copy()

NUMERIC = ["bedrooms", "bathrooms", "car_spaces", "total_rooms"]
CATEGORICAL = ["suburb", "property_type"]
FEATURES = NUMERIC + CATEGORICAL
TARGET = "sale_price"

X_dev, y_dev = dev[FEATURES], dev[TARGET]
print(f"Development: {len(dev)} rows | Holdout: {len(holdout)} rows")

```

## Cell 7

```

preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), NUMERIC),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("onehot", OneHotEncoder(handle_unknown="ignore"))]),
    CATEGORICAL),
])
models = {
    "Ridge": Ridge(alpha=10.0, random_state=RANDOM_STATE),
    "RandomForest": RandomForestRegressor(n_estimators=300,

```

```

random_state=RANDOM_STATE, n_jobs=-1),
    "GradientBoosting": GradientBoostingRegressor(random_state=RANDOM_STATE),
}
pipes = {n: Pipeline([("prep", preprocess), ("model", m)]) for n, m in
models.items()}

```

## Cell 8

```

kf = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
scoring = {"MAE": "neg_mean_absolute_error",
"RMSE": "neg_root_mean_squared_error", "R2": "r2"}
rows = []
for name, pipe in pipes.items():
    cv = cross_validate(pipe, X_dev, y_dev, cv=kf, scoring=scoring,
return_train_score=True)
    rows.append({"Model": name, "CV MAE": -cv["test_MAE"].mean(), "CV RMSE": -
cv["test_RMSE"].mean(),
                "CV R2": cv["test_R2"].mean(), "Train
R2": cv["train_R2"].mean()})
results = pd.DataFrame(rows).set_index("Model")
results["Overfit gap"] = results["Train R2"] - results["CV R2"]
results.to_csv(FIG_DIR / "cv_results.csv")
results

```

## Cell 9

```

fig, axes = plt.subplots(1, 2, figsize=(12, 4.2))
results["CV R2"].plot.bar(ax=axes[0], color="#4C72B0");
axes[0].set(title="Cross-validated R2", ylabel="R2");
axes[0].tick_params(axis="x", rotation=0)
results["CV RMSE"].div(1e6).plot.bar(ax=axes[1], color="#C44E52");
axes[1].set(title="Cross-validated RMSE", ylabel="RMSE (AUD m)");
axes[1].tick_params(axis="x", rotation=0)
fig.tight_layout(); fig.savefig(FIG_DIR / "model_comparison.png",
bbox_inches="tight"); plt.show()
best_name = results["CV R2"].idxmax()
print("Best model by CV R2:", best_name)

```

## Cell 10

```

imp_pipe = pipes[best_name].fit(X_dev, y_dev)
names = imp_pipe.named_steps["prep"].get_feature_names_out()
imp = imp_pipe.named_steps["model"].feature_importances_

agg = {}
for n, v in zip(names, imp):
    base = n.split("__")[1]
    if base.startswith("suburb_"): base = "suburb"
    elif base.startswith("property_type_"): base = "property_type"
    agg[base] = agg.get(base, 0) + v
imp_s = pd.Series(agg).sort_values(ascending=False)

```

```

fig, ax = plt.subplots(figsize=(7, 3.5))
(imp_s * 100).plot.barh(ax=ax, color="#4C72B0")
ax.invert_yaxis(); ax.set(title=f"{best_name} feature importance",
xlabel="Importance (%)")
fig.tight_layout(); fig.savefig(FIG_DIR / "feature_importance.png",
bbox_inches="tight"); plt.show()
print((imp_s * 100).round(1).astype(str) + "%")

```

## Cell 11

```

best_pipe = pipes[best_name]
oof = cross_val_predict(best_pipe, X_dev, y_dev, cv=kf)

err =
dev[["address", "suburb", "property_type", "bedrooms", "bathrooms", "car_spaces"]]
.copy()
err["actual"] = y_dev.values
err["predicted"] = np.round(oof, -3)
err["error"] = err["predicted"] - err["actual"]
err["abs_error"] = err["error"].abs()
err["pct_error"] = (err["error"] / err["actual"] * 100).round(1)

print(f"OOF performance – MAE {mean_absolute_error(y_dev, oof):.0f} | "
      f"RMSE {mean_squared_error(y_dev, oof)**0.5:.0f} | R² {r2_score(y_dev,
oof):.3f}")
worst5 = err.sort_values("abs_error", ascending=False).head(5)
worst5.to_csv(FIG_DIR / "largest_errors.csv", index=False)
worst5

```

## Cell 12

```

final_dev = pipes[best_name].fit(X_dev, y_dev)
holdout = holdout.copy()
holdout["ml_estimate"] = np.round(final_dev.predict(holdout[FEATURES]), -3)

part5 = holdout[["address", "suburb", "property_type", "bedrooms", "bathrooms",
"car_spaces", "sale_price", "ml_estimate"]].rename(columns={"sale_price": "actual
1"})
part5["llm_estimate"] = np.nan      # <- fill after querying an LLM
part5["human_estimate"] = np.nan    # <- fill with your own estimate
part5 = part5.reset_index(drop=True)
part5.to_csv(FIG_DIR / "part5_comparison_blank.csv", index=False)
print("Saved figures/part5_comparison_blank.csv – fill the two estimate
columns, then re-run the scoring cell.")
part5

```

## Cell 13

```

FILLED = Path("figures/part5_comparison_filled.csv")
if FILLED.exists():
    p5 = pd.read_csv(FILLED)
    for col in ["ml_estimate", "llm_estimate", "human_estimate"]:
        mae = (p5[col]-p5["actual"]).abs().mean()
        mape = ((p5[col]-p5["actual"]).abs()/p5["actual"]).mean()*100
        print(f"{col:15s} MAE {mae:>12,.0f} MAPE {mape:5.1f}%")

```

#### Cell 14

```

X_all, y_all = df[FEATURES], df[TARGET]
final_pipe = pipes[best_name].fit(X_all, y_all)
artifact = {"pipeline": final_pipe, "numeric": NUMERIC, "categorical":
CATEGORICAL,
            "suburbs": sorted(df["suburb"].unique().tolist()),
            "property_types": sorted(df["property_type"].unique().tolist()),
            "model_name": best_name}
joblib.dump(artifact, MODEL_DIR / "housing_model.joblib")
print("Saved model/housing_model.joblib (model:", best_name, ")")

```